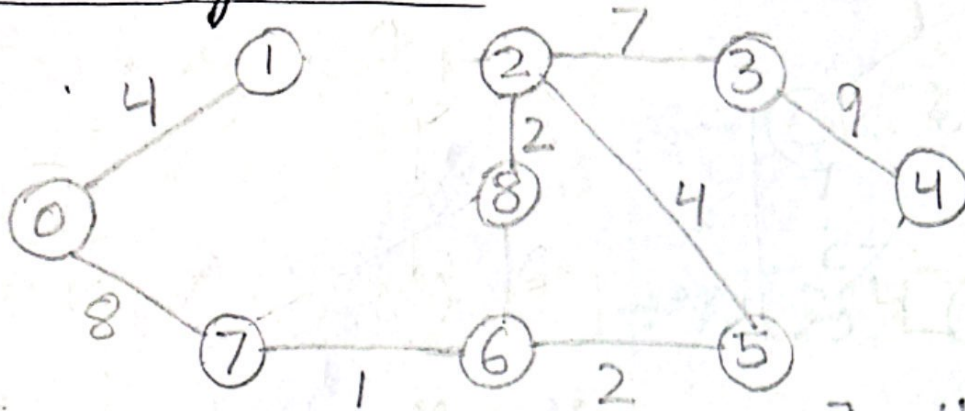


Autumn 2024

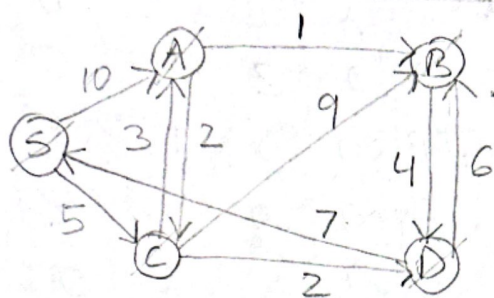
1)b) Prims Algorithm



2b)

Activity	A ₁	A ₃	A ₄	A ₆	A ₇	A ₉	A ₁₀
Start	1	3	4	8	9	11	12
Finish	3	4	7	9	11	12	14

3a)



starting vertex S

visited vertex	S	A	B	C	D
S	0	∞	∞	∞	∞
C		10	∞	5	∞
D		8	14		7
A		8	13		
B			9		

Dijkstra's algorithm does not work with negative weights because it assumes that once a node's shortest distance is found, it will not change. Negative edge weights can later provide a shorter path violating this assumption and leading to incorrect results.

b) ~~we~~ Floyd-Warshall's algorithm

$D_0 =$

	A	B	C	D
A	0	3	∞	7
B	∞	0	1	∞
C	∞	∞	0	2
D	∞	5	∞	0

$D(1) =$

	A	B	C	D
A	0	3	∞	7
B	∞	0	1	∞
C	∞	∞	0	2
D	∞	5	∞	0

$D(2) =$

	A	B	C	D
A	0	3	4	7
B	∞	0	1	∞
C	∞	∞	0	2
D	∞	5	6	0

$D(4) =$

	A	B	C	D
A	0	3	4	6
B	∞	0	1	3
C	∞	7	0	2
D	∞	5	6	0

$D(3) =$

	A	B	C	D
A	0	3	4	6
B	∞	0	1	3
C	∞	∞	0	2
D	∞	5	6	0

To determine turn (direction) at a point P_1 between points $P_0 \rightarrow P_1 \rightarrow P_2$:

$$\text{Product} = (x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)$$

If:

cross product $> 0 \rightarrow$ Left turn

$< 0 \rightarrow$ Right turn

$= 0 \rightarrow$ No turn (collinear)

4a) Given:

$$A = (1, 2)$$

$$B = (4, 5)$$

$$C = (7, 8)$$

$$D = (3, 9)$$

Let,

$$P_0 = A = (1, 2)$$

i) At location B, from $A \rightarrow B \rightarrow C$

Let,

$$P_1 = A = (1, 2)$$

$$P_2 = B = (4, 5)$$

$$P_3 = C = (7, 8)$$

$$\text{Cross product} = (x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)$$

$$= (4 - 1)(8 - 2) - (7 - 1)(5 - 2)$$

$$= 18 - 18$$

$$= 0$$

$\therefore$ The car makes no turn at location B

2) At Location C ($B \rightarrow C \rightarrow D$) turn

Let,

$$P_1 = B = (4, 5) \quad (x_1, y_1) = (4, 5) \quad (x_2, y_2) = (7, 8) \quad (x_3, y_3) = (3, 9)$$

$$P_2 = C = (7, 8)$$

$$P_3 = D = (3, 9)$$

$$\text{Cross Product} = (x_2 - x_1)(y_3 - y_1) - (x_3 - x_1)(y_2 - y_1)$$

$$= (7 - 4)(9 - 5) - (3 - 4)(8 - 5)$$

$$= 12 + 3$$

$$= 15$$

$\therefore$ The car makes left turn at location C.

4) Given,

$$P_0(1, 2), P_1(3, 4), P_2(5, 2), P_3(4, 6), P_4(7, 3), P_5(6, 1), P_6(7, 2)$$

Convex Hull

The convex hull of a set of planar points is the smallest convex polygon containing all of the points.

The point with the lowest y-value is $P_5(6, 1)$

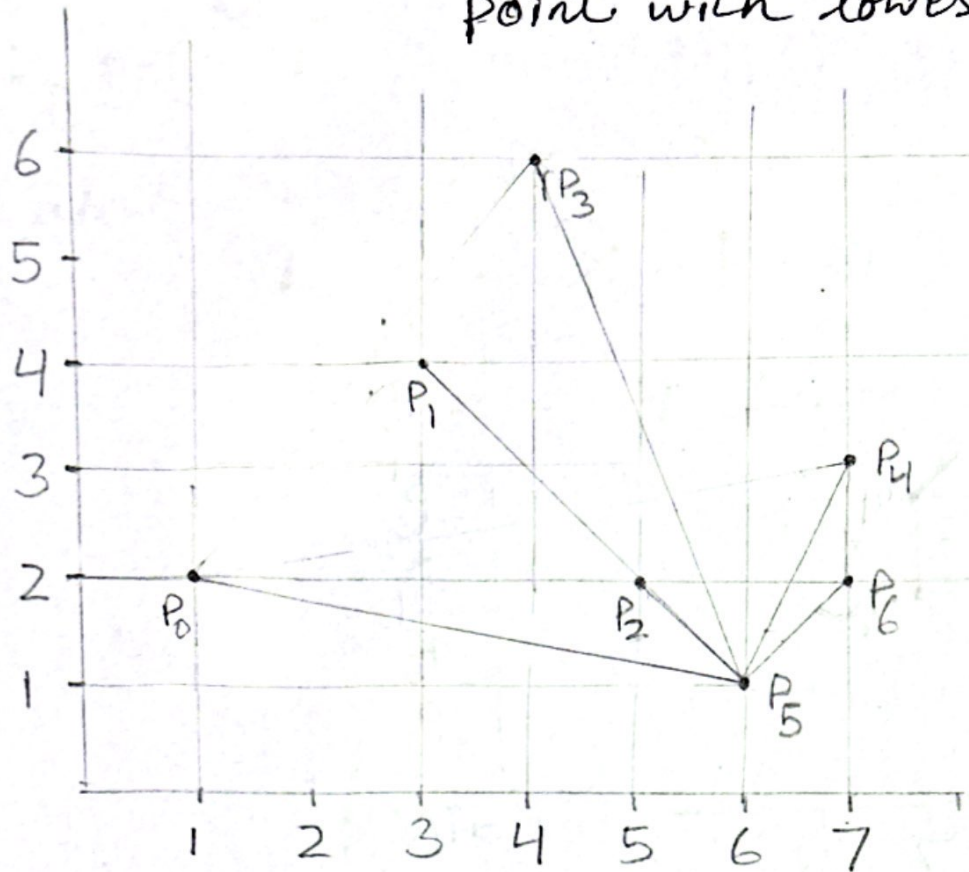
$$\text{So, } P_0 = P_5 = (6, 1)$$

stack: $[(6, 1), (7, 2)]$

Convex Hull

A convex hull is the smallest convex polygon that contains all the given points. It's like stretching a rubber band around the outermost points.

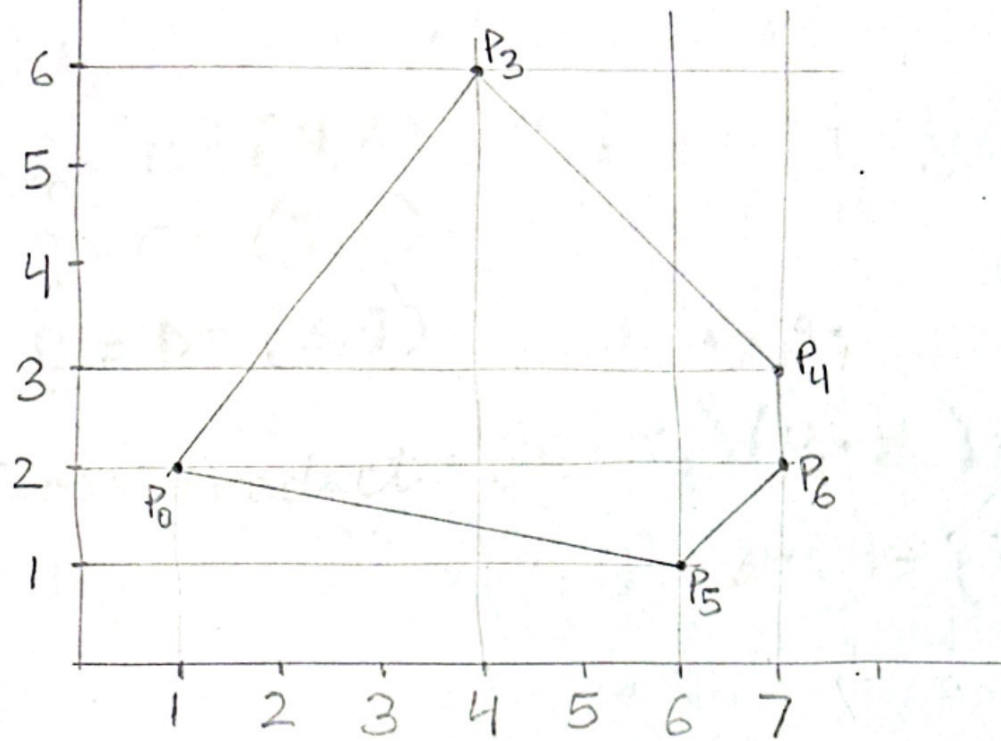
point with lowest y -value is $P_5(6,1)$



Sorted order by angle:

$P_6(7,2)$, $P_4(7,3)$, $P_3(4,6)$, $P_1(3,4)$, $P_2(5,2)$, $P_0(1,2)$

(1.2) Convex Hull by Jarvis's Algorithm



P_0
P_1
P_2
P_3
P_4
P_6
P_5

STACK

5) a) The N-Queen Problem is to place n queens on an $n \times n$ chessboard such that no two queens attack each other, meaning no two queens share the same row, column, or diagonal.

Backtracking Algorithm steps:

1. Place a queen in the first row
2. Try to place a queen in each column of the next row.
3. Before placing, check:
 - * No other queen is in the same column
 - * No queen lies on the same diagonal.
4. If valid, proceed to the next row.
5. If no column is valid, backtrack to the previous row and try the next option.

4 Queens

	Q ₁		
			Q ₂
Q ₃			
		Q ₄	

[2, 4, 1, 3]

5b) P (Polynomial Time):

Problems that can be solved in polynomial time by a deterministic algorithm.

Example: Matrix determinant

NP (Nondeterministic Polynomial Time):

Problems for which a given solution can be verified in polynomial time, even if we don't know how to find the solution quickly.

Example: graph isomorphism

NP-Hard

A problem Y is NP-Hard if every problem X in NP can be reduced to Y in polynomial time.

Example: Halting problem

NP-Complete:

A problem Y is NP complete if:

1. Y is in NP
2. Y is NP-Hard

Example: Hamiltonian cycle

Relationship Diagram:

2. P (Polynomial Time)

- * $P \subseteq NP$
- * $NP\text{-complete} \subseteq NP$
- * $NP\text{-complete} \subseteq NP\text{-Hard}$
- * If $P = NP$, then all NP problems can be solved in polynomial time.

Problems for which a given solution can be verified in polynomial time, even if we don't know how to find the solution quickly.

Example: Graph isomorphism

NP-Hard

A problem Y is NP-Hard if every problem X in NP can be reduced to Y in polynomial time.

Example: Halting problem

NP-complete

A problem Y is NP-complete if:

1. Y is in NP

2. Y is NP-Hard

Example: Hamiltonian cycle